

Project Information

Project	Threat Detection Platform
Week	Week 2 — Risk Scoring + Threat Normalization
Data Flow	Python + MongoDB + Threat Intelligence
Stack	Python + MongoDB + Threat Intelligence

Week 2 Objectives

- By the end of Week 2, the Threat Detection Platform should be able to:
 - analyze collected threat records
 - assign risk levels based on malicious score
 - normalize stored threat intelligence
 - enrich raw threat data with security context
 - generate structured dashboard-ready threat output

DAY 6**Risk Scoring Engine Setup****STEP 1 — Create Risk Analysis Module (Goal: Assign risk levels to threats)**

```
from db import collection
def calculate_risk(score):
    if score >= 5:
        return 'HIGH'
    elif score >= 2:
        return 'MEDIUM'
    else:
        return 'LOW'
```

Expected Output: Risk scoring logic ready

STEP 2 — Update Risk Levels in MongoDB (Goal: Add severity labels)

```
def update_risk_levels():
    threats = collection.find()
    for threat in threats:
        risk_level = calculate_risk(threat['malicious_score'])
        collection.update_one({'_id': threat['_id']}, {'$set': {'risk_level':
risk_level}})
    print(f"Updated {threat['ip']} -> {risk_level}")
```

```
update_risk_levels()
```

Expected Output: Updated 8.8.8.8 -> LOW

DAY 7

Threat Normalization

STEP 3 — Create Normalization Module (Goal: Standardize threat schema)

```
from db import collection
def normalize_threats():
    threats = collection.find()
    for threat in threats:
        normalized = {
            'ip': threat.get('ip'),
            'malicious_score': threat.get('malicious_score', 0),
            'risk_level': threat.get('risk_level', 'LOW'),
            'timestamp': threat.get('timestamp')
        }
        collection.update_one({'_id': threat['_id']}, {'$set': normalized})
        print(f"Normalized: {threat['ip']}")

normalize_threats()
```

Expected Output: Normalized: 8.8.8.8

DAY 8

Dashboard Data Preparation

STEP 4 — Create Dashboard Export Module (Goal: Prepare dashboard-ready output)

```
from db import collection
def export_dashboard_data():
    threats = collection.find({}, {'_id': 0})
    for threat in threats:
        print(threat)
export_dashboard_data()
```

Expected Output: Clean dashboard-ready threat records

DAY 9

Pipeline Execution + Validation

STEP 5 — Execute Week 2 Modules (Goal: Validate full Week 2 pipeline)

```
python risk_analysis.py
python normalize_data.py
python dashboard_data.py
```

Expected Output:

Updated 8.8.8.8 -> LOW

Normalized: 8.8.8.8

{'ip': '8.8.8.8', 'malicious_score': 0, 'risk_level': 'LOW'}

DAY 10**Threat Intelligence Output Verification****STEP 6 — Validate Structured Threat Intelligence (Goal: Confirm Week 2 success)**

Verify MongoDB records now contain:

- ip
- malicious_score
- risk_level
- timestamp

Expected Output: Structured threat intelligence ready for Week 3

Completion Checklist

- ✓ Risk scoring logic created
- ✓ Risk levels added to MongoDB
- ✓ Threat records normalized
- ✓ Dashboard-ready output prepared
- ✓ Week 2 scripts executed successfully
- ✓ Structured threat intelligence generated

Week 2 — Required GitHub Commits**Commit Message**

```
git commit -m "feat: add risk scoring engine"
git commit -m "feat: add threat normalization module"
git commit -m "feat: add dashboard export module"
git commit -m "test: verify week 2 threat intelligence pipeline"
```